WEEKLY MINI PROJECT 04 · TEAM 3

Bytes for Futures Assessment Platform

Work Package · Frontend B

Frontend | React + TS | Monaco

PROJECT	TEAM LEADER	OWNER
Bytes for Futures Assessment Platform	**Ali Mikdad**	**Maya Al Hamwi**

ROLE	AREA
Frontend developer	**My Tests · Test Intro · MCQ · Coding · My Results · Exam guard wiring**

> *Every screen a student uses, from the list of tests to the result. Every screen shows only what the backend returns.*

> **Use the project pattern:** `pages/` for screens, `context/` for server data, `services/` for API calls (only contexts call them), `types/` copied from `API.md`. Pages never call `fetch`, and never decide scores, locks or time.

> **Full reference:** this package is a summary of `docs/TASKS.md`. Every name, endpoint, shape and message is fixed in `docs/API.md` and `docs/PROJECT-STRUCTURE.md`. If this package and the docs ever disagree, the docs win; tell Ali.

1 Your Work at a Glance

#	Task	Main file(s)
T1	Clock and countdown (first small PR)	`utils/serverClock.ts`, `components/common/Countdown.tsx`
T2	Attempt session key	`utils/attemptSession.ts`, `services/student.api.ts`
T3	Student contexts	`context/StudentTestsContext.tsx`, `context/AttemptContext.tsx`
T4	My Tests	`pages/student/MyTests/`
T5	Test Intro (pledge + full screen)	`pages/student/TestIntro/`
T6	MCQ test with autosave	`pages/student/Attempt/McqTestView.tsx`, `QuestionCard.tsx`
T7	Code editor	`editor/CodeEditor.tsx`, `editor/CodeEditor.css`
T8	Coding test: Run	`CodingTestView.tsx`, `ProblemPanel.tsx`, `RunOutput.tsx`
T9	Coding test: Submit and Finish	`SubmitResultBox.tsx`, `ProblemSidebar.tsx`, `AttemptResult.tsx`

#	Task	Main file(s)
T10	Exam integrity on the attempt page	`AttemptPage.tsx`, `IntegrityStatus.tsx`
T11	My Results	`pages/student/MyResults/`

2 Work Without Waiting

PHASE 0 · DAY 1 MORNING

Stubs everywhere

Ali and Esmaeel push both skeletons. Every endpoint answers with the example JSON from `API.md`; every route shows a placeholder.

PHASE 1 · DAYS 1–2

Build alone

Everyone works in their own files, against the fixed names. Replace stubs only inside your own module.

PHASE 2 · END OF DAY 2

Real integration

Every stub replaced, every context on the real endpoints, every message checked against `API.md`.

Your work depends on `docs/API.md` and `docs/PROJECT-STRUCTURE.md`, not on someone's unfinished code. If a dependency is not merged yet, code against the fixed name and test with the stub.

3 Your Endpoints (used through the contexts)

```
GET  /api/student/tests                GET /api/student/tests/:testId
POST /api/student/tests/:testId/attempts
GET  /api/student/attempts/:attemptId
PUT  /api/student/attempts/:attemptId/mcq-answers/:questionId
POST /api/student/attempts/:attemptId/mcq-submit
POST /api/student/attempts/:attemptId/problems/:problemId/run
POST /api/student/attempts/:attemptId/problems/:problemId/submit
POST /api/student/attempts/:attemptId/finish
POST /api/student/attempts/:attemptId/violations
GET  /api/student/results
```

Every call under `/student/attempts/:attemptId` sends `X-Attempt-Session` (`API.md` 19.4).

4 Your Tasks, Step by Step

T1 Clock and countdown (first small PR)

`client/src/utils/serverClock.ts` · `client/src/components/common/Countdown.tsx (+ css)`

Function	What it does
`getOffsetMs(serverNow)`	`Date.parse(serverNow) - Date.now()`
`getRemainingMs(deadline, offsetMs)`	`max(0, Date.parse(deadline) - (Date.now() + offsetMs))`
`formatRemaining(ms)`	`754000` → `12:34`

`Countdown({ deadline, serverOffsetMs, onTimeUp })`:

1. Updates every second with `setInterval` (cleared on unmount).
2. Color: normal, `--color-warning` under 5 minutes, `--color-error` under 1 minute.
3. Calls `onTimeUp` **once** when it reaches `00:00`.
4. Never stores its own start time: it always calculates from `deadline`.

✔ Done when: start a test, refresh at any moment → the countdown continues from the same deadline.

T2 Attempt session key

`client/src/utils/attemptSession.ts` · `client/src/services/student.api.ts`

1. `saveAttemptSession(attemptId, key)`, `getAttemptSession(attemptId)`: `sessionStorage`, key `b4f.attemptSession.<attemptId>`, every call in `try/catch`.
2. `startAttempt` saves the returned `sessionKey`.
3. Every attempt call in `student.api.ts` sends `X-Attempt-Session`.
4. My Tests **Continue** calls `await enter()`, then `startAttempt(testId, true)` (resume; the backend needs `acceptedRules: true` here too) before opening the attempt.
5. Session `409`: set `isSessionReplaced`, hide the questions, show `This attempt is open in another window or device.` and **Continue here** (resume).

✔ Done when: opening the same attempt in a second window makes the first one show the session message.

T3 Student contexts

`context/StudentTestsContext.tsx` · `context/AttemptContext.tsx` · `types/student.types.ts` · `types/execution.types.ts`

Copy the pattern from `AuthContext`. Fixed shapes are in section 5 of this package.

1. `StudentTestsContext`: `loadTests`, `loadTest`, `loadResults`.
2. `AttemptContext`: `startAttempt`, `loadAttempt`, `saveMcqAnswer`, `submitMcq`, `runCode`, `submitCode`, `finishCoding`, `reportViolation`.
   - Every attempt response updates `serverOffsetMs`.
   - `runCode` saves the result in `runResults[problemId]` and updates that problem's `runCount` / `runsRemaining` in `attempt`.
   - `submitCode` saves `submitResults[problemId]` and sets the problem's `submission`.
   - Any `409` with the time-over or terminated message → reload the attempt.
   - A new action clears the old `actionError`.

✔ Done when: after every action, states and counters update without a page reload.

T4 My Tests (`/student/tests`)

`pages/student/MyTests/`

1. `TestCard` per test, in order: title, type, duration, item count, `StateBadge`.
   - `LOCKED`: `lockReason`, no button.
   - `AVAILABLE`: **Open** → Test Intro.
   - `IN_PROGRESS`: **Continue** → `await enter()`, then `startAttempt(testId, true)` (resume, new session key), then `/student/attempts/:attemptId`.
   - `SUBMITTED`: the score. `EXPIRED`: `Expired — not submitted`.
   - `TERMINATED`: `Terminated — exam rules violated` and the score.
2. Empty view: `No tests are available yet.`

✔ Done when: My Tests shows every state, with the lock reason for locked tests.

T5 Test Intro (`/student/tests/:testId`)

`pages/student/TestIntro/`

1. Title, duration, item count, rules (one attempt; the timer starts when you press Start; refreshing does not stop it; full screen is required; leaving full screen or switching tabs twice ends the exam; copy or paste three times ends the exam).
2. The pledge checkbox (`API.md` 19.6). Start disabled until it is ticked.
3. If `useFullscreen().isSupported` is false, show `This exam needs full screen. Please use a computer with Chrome, Edge or Firefox.` and keep Start disabled.
4. **Start test**: `await enter()` first, then `startAttempt(testId, true)` (loading while it runs).
5. Direct address of an unpublished or locked test → the backend `403` message and a link back to My Tests. Never a blank page.

✔ Done when: the test starts only in full screen, after the pledge; an unpublished test opened by address shows the `403` message.

T6 MCQ test with autosave (`McqTestView`)

`pages/student/Attempt/McqTestView.tsx` · `QuestionCard.tsx`

1. `Countdown` at the top (sticky).
2. Every `QuestionCard` : text and radio options. Choices start from `selectedOptionId` , so a refresh keeps them. Choices are kept in `AttemptPage` state (outside `FullscreenGate`) and passed to `McqTestView` as props, so leaving full screen does not clear them.
3. Every choice calls `saveMcqAnswer(questionId, optionId)` . Show `Saved` / `Saving…` / `Not saved, retrying` per question. A failed save is retried after 2 s (max 3 times); the final **Submit** still sends every choice.
4. Counter: `12 of 20 answered` .
5. **Submit answers** → `ConfirmDialog` (`You have 3 unanswered questions. Unanswered questions count as wrong. Submit anyway?`) → `submitMcq` .
6. After submit: `AttemptResult` with `16 / 20 correct (80%)` and a link to My Tests.
7. `onTimeUp` : all radios and the button disabled, message `Time is over.` , then after `GRACE_SECONDS` reload the attempt and show `Expired — not submitted` .

✔ Done when: the MCQ test submits once and shows the backend result; choices come back after a refresh.

T7 Code editor

`client/src/editor/CodeEditor.tsx` · `CodeEditor.css`

1. Uses the bundled Monaco from `monacoSetup.ts` (Ali). Language `javascript` , a dark theme, line numbers, no minimap.
2. `errorLine` → `monaco.editor.setModelMarkers` on that line with the error message. Markers are cleared on the next change.
3. `isReadOnly` after the problem is submitted or the time is over.
4. Starter code: `function <functionName>(/* parameters */) {\n \n}\n` .
5. `data-exam-clipboard="allow"` and a `ref` (`wrapperRef`) on the container; in `onMount` : `guardRef.current?.dispose()` , then `attachEditorPasteGuard(editor, onViolation, wrapperRef.current)` ; `dispose()` on unmount.
6. If the instructor refuses Monaco: the same component renders a `<textarea>` with the same props.

✔ Done when: the editor gives suggestions and live syntax errors, with no CDN request.

T8 Coding test: Run (`CodingTestView`)

`CodingTestView.tsx` · `ProblemPanel.tsx` · `RunOutput.tsx` · `ExecutionModeNotice.tsx`

```
Layout (on a phone the sidebar becomes a select above the panel)
┌──────────────────────────── Countdown · ExecutionModeNotice · Finish test ┐
│ ProblemSidebar │ ProblemPanel                                             │
│  1. Two Sum ✓  │  title · points · description                            │
│  2. Reverse    │  function name · sample input → expected output          │
│                │  ┌ CodeEditor (Monaco) ──────────────────────────────┐   │
│                │  └──────────────────────────────────────────────────┘   │
│                │  [Run (17 left)]  [Submit]                               │
│                │  RunOutput / SubmitResultBox                             │
└────────────────┴──────────────────────────────────────────────────────────┘
```

1. **Run (N left)** calls `runCode(problemId, code)`. Disabled while running, when `runsRemaining` is 0, after submit, and after time is up.
2. `RunOutput` shows the result by `status` (table below).
3. `logs` (the student's `console.log`) show in a separate **Console** box under the result, never mixed with the error.
4. Values are shown with `JSON.stringify(value)` so `"1"` and `1` look different.
5. `503` → the backend message (`...Your run was not counted...`) and the Run button stays enabled.
6. `ExecutionModeNotice` is always visible: `MOCK` → `Mock execution: your code is not really executed. Run shows a simulated result.` `JUDGE0` → `Your code runs on an external execution service.`
7. The code of each problem is kept in state **in `AttemptPage`** and passed to `CodingTestView` as props (`codeMap`, `onCodeChange`), so switching problems **and leaving full screen** never lose it. `FullscreenGate` unmounts its children, so state inside `CodingTestView` would be lost.

Status	Shows
`PASSED`	green title + message, input, expected output, your output
`WRONG_ANSWER`	orange title + message, input, expected, your output (side by side)
`SYNTAX_ERROR` / `RUNTIME_ERROR`	red title, `error.type: error.message`, `Line <n>` (when not null), and the line marked in the editor
`FUNCTION_NOT_FOUND`	red title + message (it tells the exact name to use)
`TIME_LIMIT_EXCEEDED`	red title + message
`MOCK`	the `[MOCK]` message

✔ Done when: Run shows a readable result for every status, with the error line marked in the editor and `console.log` in its own box; the 21st run is impossible from the UI.

T9 Coding test: Submit and Finish

`SubmitResultBox.tsx` · `ProblemSidebar.tsx` · `AttemptResult.tsx`

Submit

1. **Submit** → `ConfirmDialog` (`You can submit this problem only once. Submit now?`) → `submitCode`.
2. `SubmitResultBox`: `4 of 5 tests passed` and the points. The editor becomes read-only. The sidebar shows ✓.
3. `409` (already submitted) → message, and reload the attempt.

Finish

1. **Finish test** → `ConfirmDialog` that lists unsubmitted problems (`Unsubmitted problems score 0.`).
2. After finish: `AttemptResult` with the coding score and the per-problem points.
3. `onTimeUp`: everything read-only, `Time is over.`, reload after `GRACE_SECONDS`.

✔ Done when: Submit works once per problem and shows `X of Y tests passed`; Finish shows the coding score.

T10 Exam integrity on the attempt page

`pages/student/Attempt/AttemptPage.tsx` · `IntegrityStatus.tsx`

Rules, limits and messages: `API.md`, section 19. Toolkit: Ali's `exam-guard/` (see section 5 of this package for the wiring).

1. Wrap the view in `FullscreenGate`, guard it with `useExamGuard`, show `ViolationWarningDialog` and `IntegrityStatus` (`Warnings: focus 1/2 · copy/paste 0/3`). `ViolationWarningDialog` goes **outside** `FullscreenGate`, or the warning disappears when the student leaves full screen.
2. `reportViolation(type)`: calls the endpoint, updates `integrity`, sets `lastWarning`. If `terminated: true`: set the attempt to the returned result, call `exit()`, show `AttemptResult`. A failed report is retried once and never blocks the page.
3. **TERMINATED everywhere:** `AttemptResult`, My Tests card and My Results row show `Terminated — exam rules violated`, the reason (on the result screen) and the score.
4. After `SUBMITTED`, `EXPIRED` or `TERMINATED`: `isActive` is false, and the page calls `exit()`.

✔ Done when: every warning from the backend is shown; a termination ends the screen at once and shows the reason and the score; the content is hidden outside full screen.

T11 My Results (`/student/results`)

`pages/student/MyResults/`

1. `loadResults()` when the page opens.
2. One row per finished test: title, `StateBadge`, score with `formatScore` (terminated: the score and `Terminated — exam rules violated`), submit or termination time.
3. Empty view: `You have not finished any test yet.`

✔ Done when: My Results shows every finished test with the right label.

5 Fixed Shapes and Shared Files

client/src/context/*.tsx (fixed value shapes you own)

```
// StudentTestsContext (Maya)
tests: StudentTestCard[];
isTestsLoading: boolean;
testsError: string | null;
loadTests(): Promise<void>;
currentTest: StudentTestCard | null;
isTestLoading: boolean;
testError: string | null;          // includes the 403 message for an unpublished test
loadTest(testId: number): Promise<void>;
results: StudentResult[];
isResultsLoading: boolean;
resultsError: string | null;
loadResults(): Promise<void>;

// AttemptContext (Maya)
attempt: StudentAttempt | null;
isAttemptLoading: boolean;
attemptError: string | null;
startAttempt(testId: number, acceptedRules: boolean): Promise<number | null>;  // new or resume; saves sessionKey;
null if it failed
loadAttempt(attemptId: number): Promise<void>;

serverOffsetMs: number;            // serverNow - Date.now(), updated on every attempt response

isSubmitting: boolean;
actionError: string | null;
submitMcq(answers: McqAnswerPayload[]): Promise<boolean>;
saveMcqAnswer(questionId: number, optionId: number | null): Promise<boolean>;   // autosave, quiet on success
savingQuestionIds: number[];       // shows "Saving..." next to a question

runResults: Record<number, RunResult>;          // by problemId, last run only
submitResults: Record<number, SubmitResult>;    // by problemId
runningProblemId: number | null;
runCode(problemId: number, code: string): Promise<boolean>;
submitCode(problemId: number, code: string): Promise<boolean>;
finishCoding(): Promise<boolean>;
clearActionError(): void;

integrity: AttemptIntegrity | null;             // counts and limits from the backend
lastWarning: string | null;                     // backend warning text, shown in ViolationWarningDialog
reportViolation(type: ClientViolationType): Promise<void>;
clearWarning(): void;
isSessionReplaced: boolean;                     // true after the "open in another window" 409
```

```
// client/src/editor/CodeEditor.tsx  (props fixed)
value: string;
onChange: (value: string) => void;
isReadOnly?: boolean;
errorLine?: number | null;     // from RunResult.error.line
height?: string;
onViolation?: (type: 'PASTE_EXTERNAL' | 'DROP_EXTERNAL') => void;   // set while the attempt is IN_PROGRESS
```

Exam-guard wiring (PROJECT-STRUCTURE.md 16.3) — the toolkit is Ali's, do not edit it

```
const { isSupported, isFullscreen, enter, exit } = useFullscreen();
const isActive = attempt?.status === 'IN_PROGRESS' && !isTimeUp;

useExamGuard({ isActive, onViolation: reportViolation });

<>
  <FullscreenGate isFullscreen={isFullscreen} onEnter={enter}>
    {/* McqTestView or CodingTestView — their state (code, choices) lives here in AttemptPage */}
  </FullscreenGate>
  {/* OUTSIDE the gate: leaving full screen is itself a violation, so the warning must stay visible then. */}
  <ViolationWarningDialog warning={lastWarning} onReturn={() => { clearWarning(); enter(); }} />
</>
```

You also import (do not edit): `components/common/*`, `http.ts` `request<T>()`, `ApiError`, `useAuth()`, `formatScore()` from Ali.

6 Rules for You

- Only edit your own files (`TASKS.md` section 4).
- Do not count violations, decide warnings, or decide termination (the backend does).
- Do not add your own full-screen, visibility, copy or paste listeners (use `exam-guard/`).
- Do not calculate scores, points, remaining runs, or locks.
- Do not start a timer from the click time (always from `deadline`).
- Do not show anything about hidden tests except the counts.
- Do not call `fetch` or services inside pages or components. Do not write backend code.
- Do not change routes (ask Ali). Do not use hardcoded colors.
- CSS uses the `my-tests-`, `test-intro-`, `attempt-`, `my-results-`, `code-editor-`, `countdown-` prefixes and theme colors only. Pages work on desktop and phone.

7 Presentation Part

Your parts in the English presentation: **the main flow demo** (instructor publishes → student takes tests in order → scores, with loading, empty, error and expired states), and **exam integrity live** with Esmaeel (leave full screen, switch tab, the violation log). Prepare your part and one rule from another member's area, and be ready to explain your own code.